

Разработка механизма кэширования постраничного индекса LSM-дерева на основе B-деревьев



студент: Антон Кузнец
руководитель: Константин Осипов

LSM

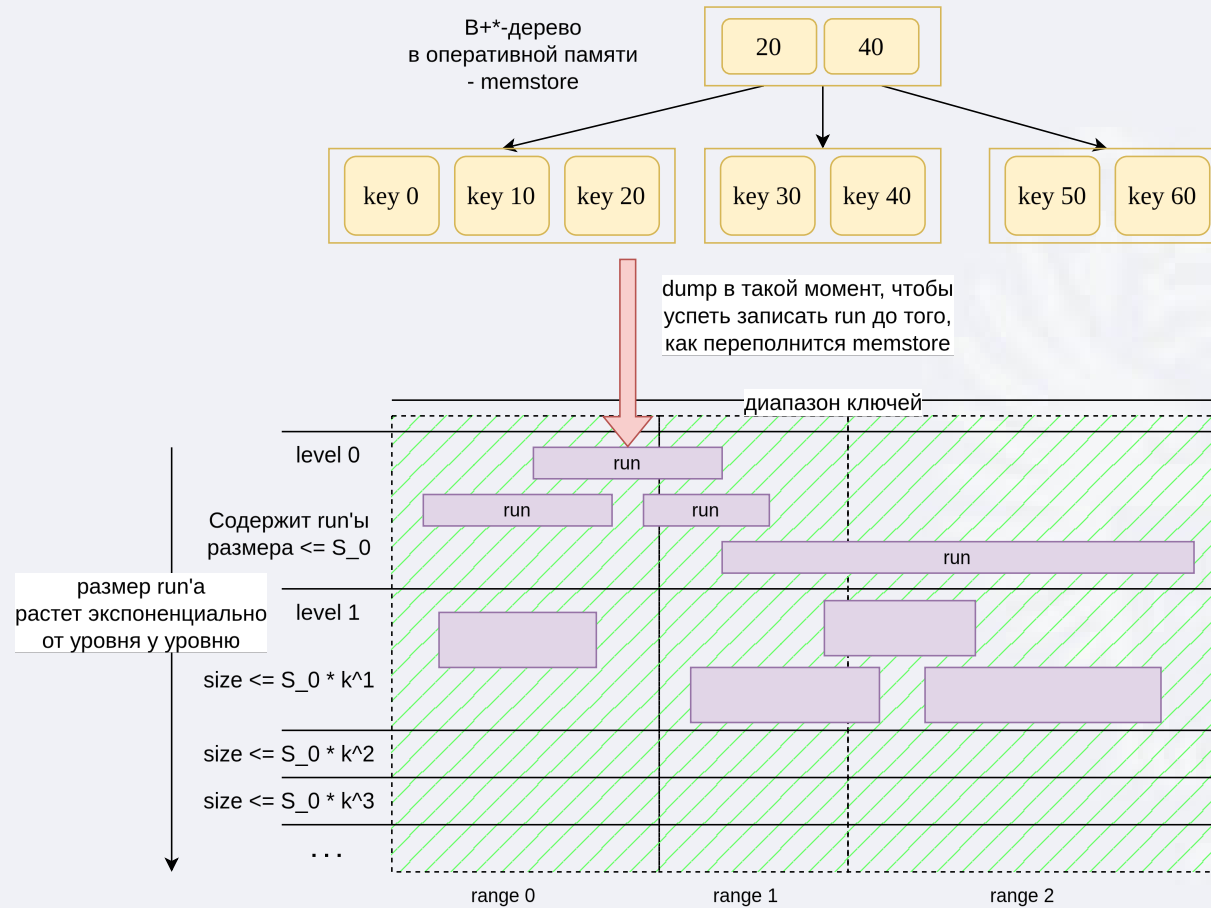
LSM-tree как идея был предложен в середине 1990-х.

Опубликован в статье *Patrick O'Neil, Edward Cheng, Dieter Gawlick, Elizabeth O'Neil, «The Log-Structured Merge-Tree (LSM-Tree)», 1996, Acta Informatica.*

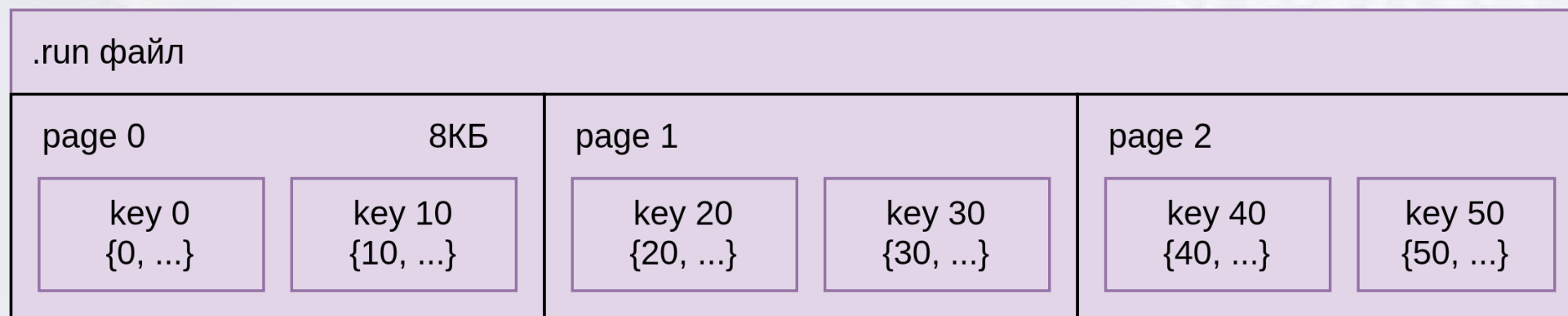
Первое широкое промышленное распространение LSM-подход получил позже, в системах типа Google Bigtable (2000-е), а затем HBase, Cassandra, LevelDB, RocksDB и т.д.



LSM (Vinyl)



LSM (Vinyl) .run файл



кортежи отсортированы по
возрастанию ключа

LSM

Зачем нужна компактификация:

- новые записи дешево писать в память и периодически сбрасывать в новые маленькие гуп'ы
- но если просто накапливать много гуп'ов, чтение станет дорогим, потому что придётся проверять слишком много гуп'ов

Поэтому LSM, и в частности Vinyl, балансирует между тем чтобы:

- сделать запись дешевой
- не дать чтению деградировать из-за большого числа гуп'ов

Для этого периодически несколько старых гуп'ов сливаются в один новый, чтобы уменьшить read amplification.





Стратегия компактификации в LSM балансирует между несколькими свойствами:

- Write amplification

Чем более агрессивно происходит компактификация, тем больше одни и те же данные перезаписываются. Было бы плохо переписывать большие объёмы данных слишком часто.

- Read amplification

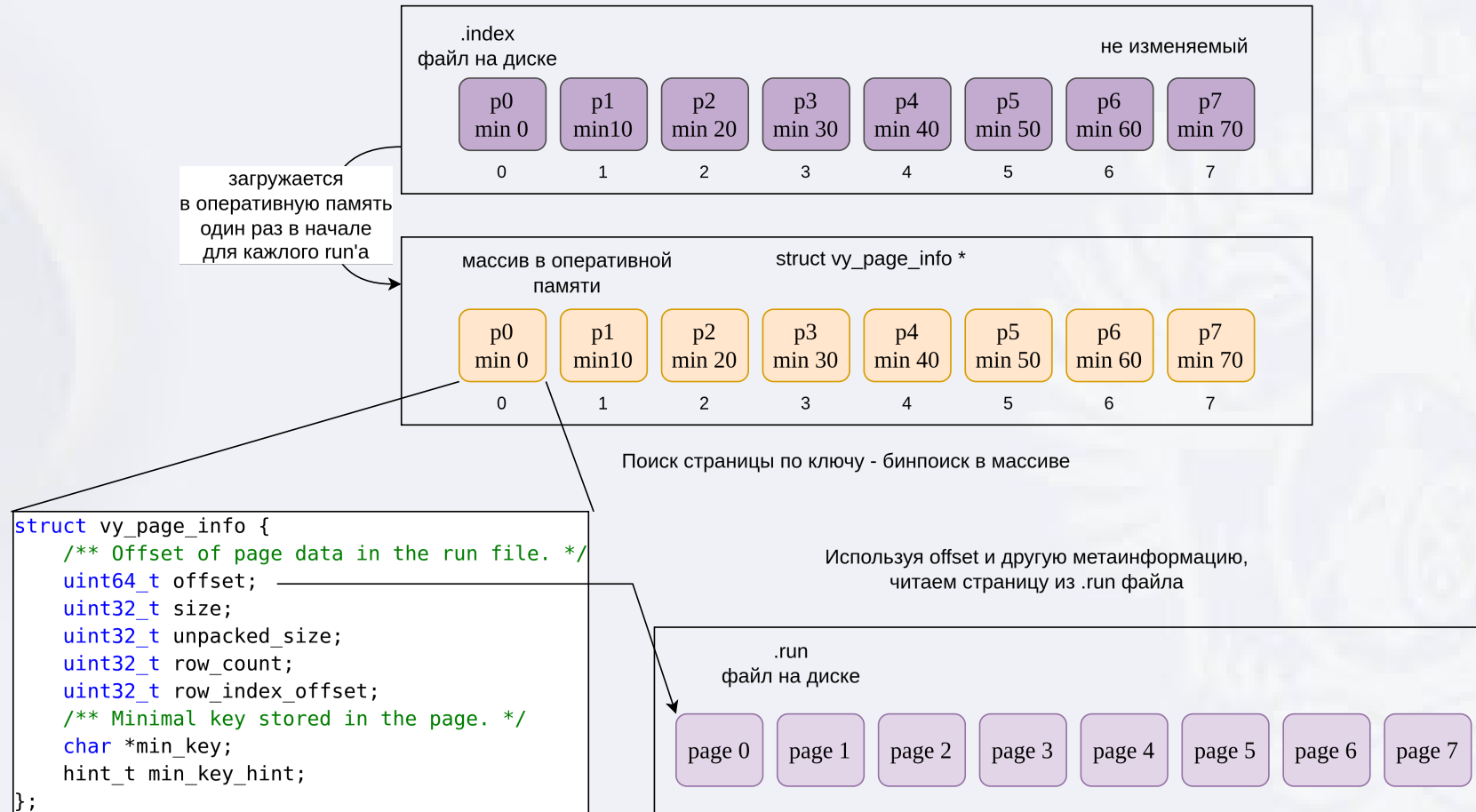
Если не compact-ить совсем, то становится много слоёв из данных `logs`, в том числе уже не актуальных, и чтению приходится проверять много источников.

- Space amplification

Пока старые версии ключей и tombstones не схлопнуты, на диске лежит много «мёртвых» данных.

- предсказуемость фоновой нагрузки

Постраничный индекс в оперативной памяти



Постраничный индекс в оперативной памяти



page_size в Vinyl по умолчанию 8 КБ.

Одна struct vy_page_info без учёта min_key занимает примерно 40 Б.

$40 / 8192 = 0.0048$ (0.5% от объёма данных)

Если добавить, что у каждой страницы ещё отдельно хранится min_key по указателю, то реалистичнее считать не 40 Б, а примерно 56–72 Б на страницу для коротких/средних ключей. Тогда overhead получается уже порядка 0.68%–0.88%, а с длинными ключами может уйти и к 1%+.

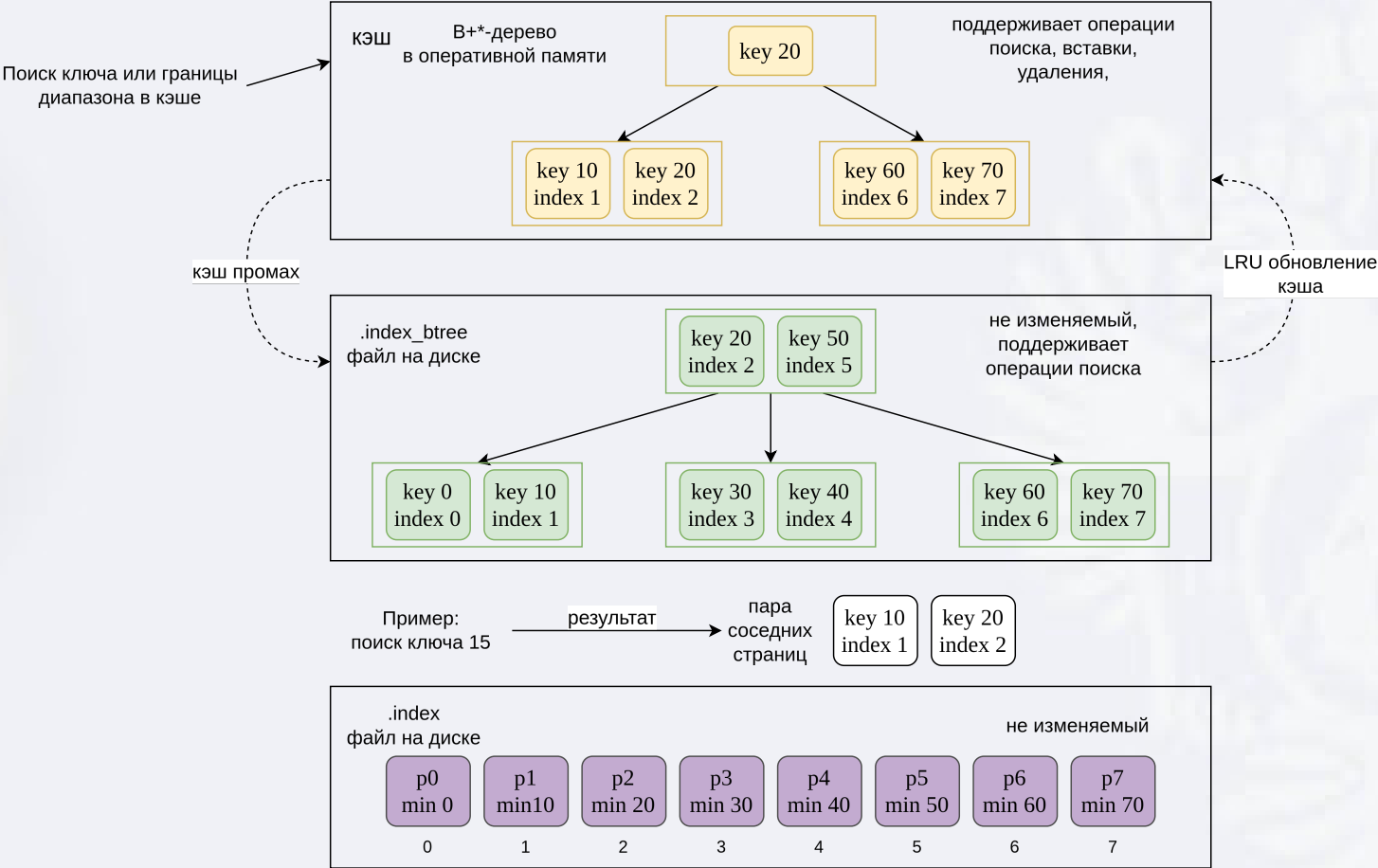
Проблема: Для терабайтного набора данных это сотни мегабайт – гигабайты одних только метаданных в оперативной памяти.

Задачи

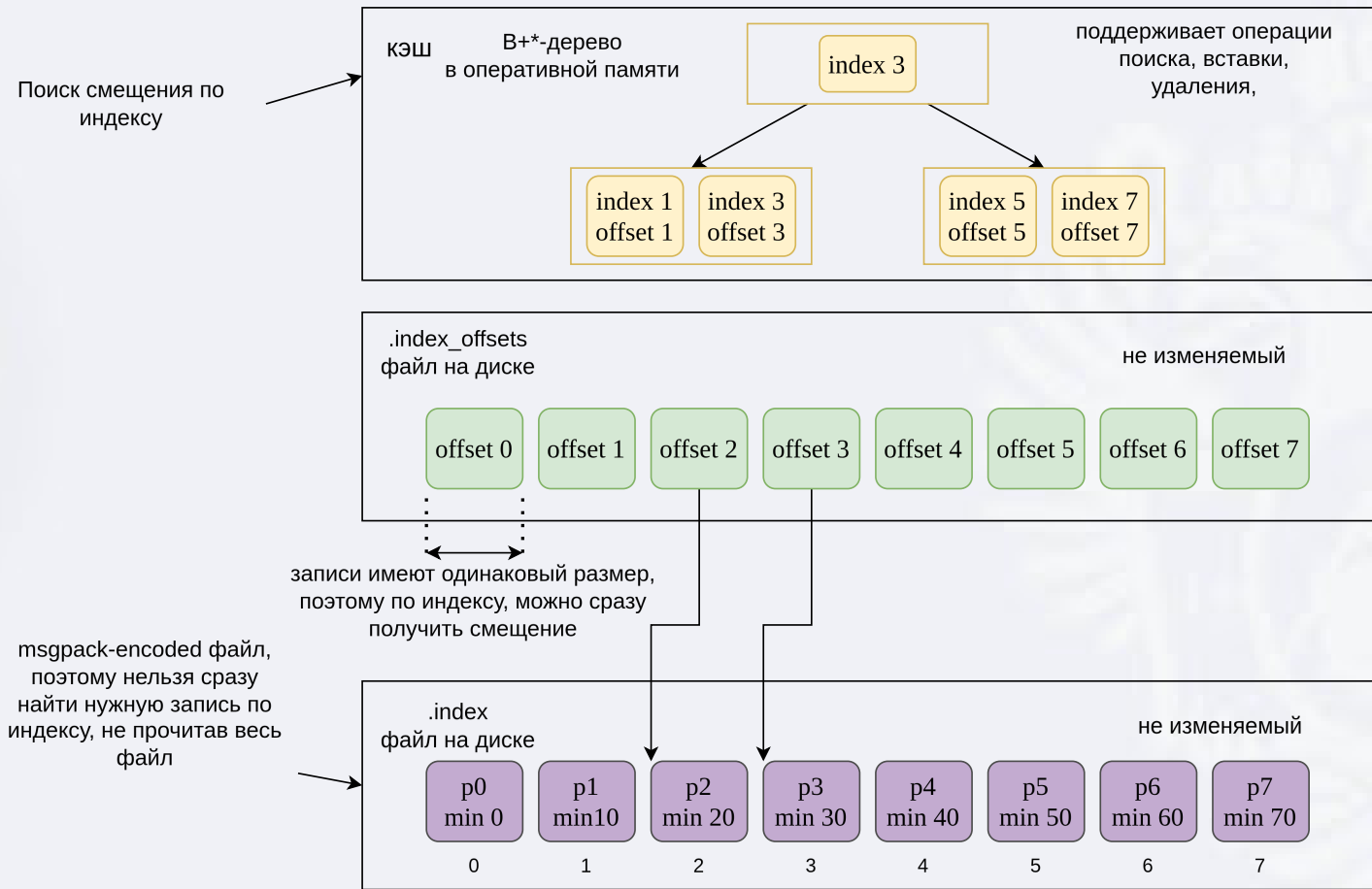


- Проанализировать устройство LSM-деревьев и особенности реализации Vinyl в Tarantool.
- Исследовать существующий постраничный индекс Vinyl и определить ограничения его масштабирования по памяти.
- Сформулировать требования к новому механизму постраничного индекса.
- Разработать структуру постраничного индекса с размещением основной части метаданных на диске и ограниченным рабочим набором в памяти.
- Реализовать предложенный подход в кодовой базе Tarantool.
- Сравнить разработанное решение с in-методу подходом.
- Провести экспериментальную оценку по памяти и производительности.

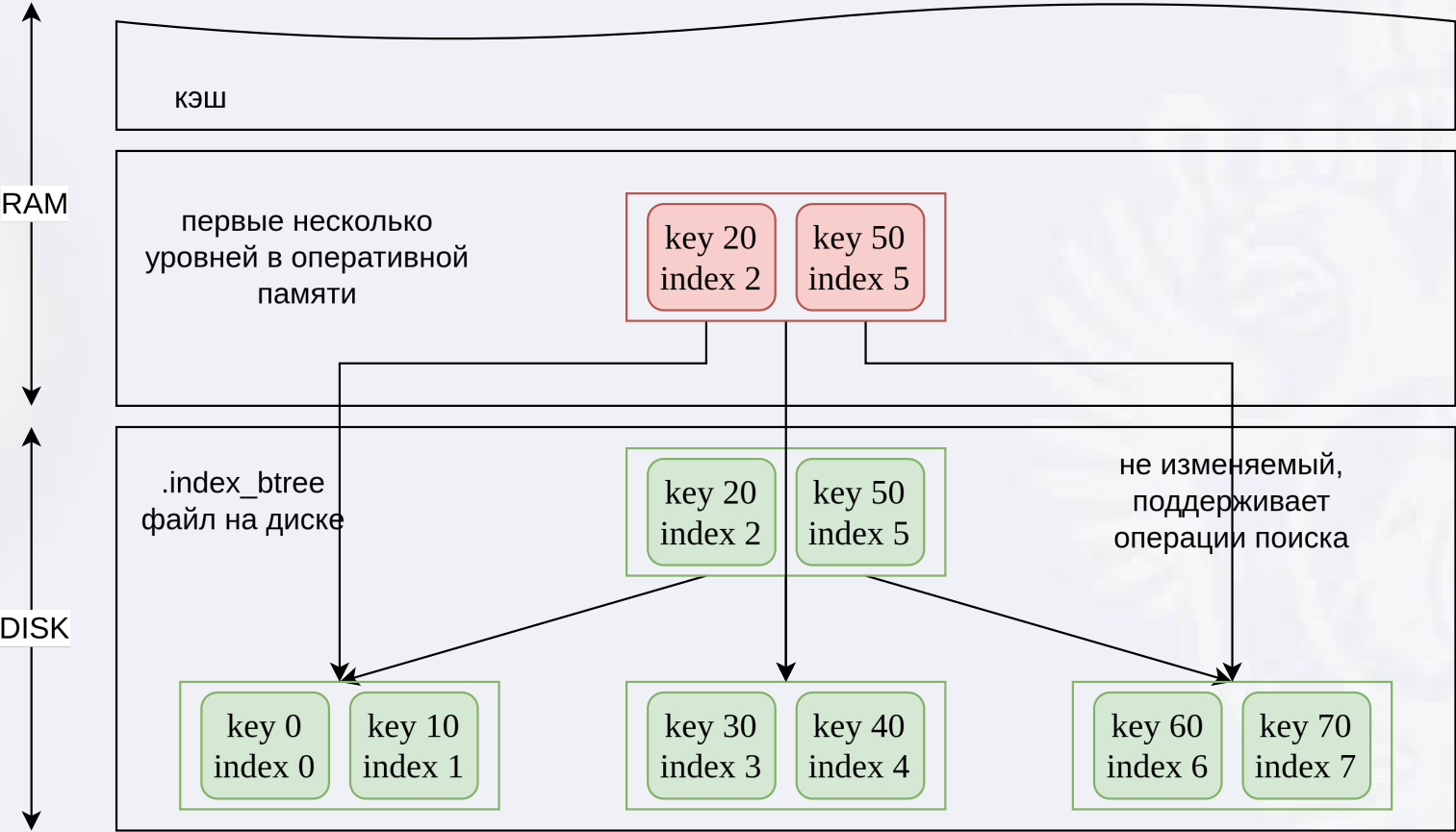
Индекс страницы по ключу



Страница по индексу



В-дерево



Бенчмаркинг: методика и метрики



YCSB (Yahoo! Cloud Serving Benchmark)-подобные нагрузки на наборе данных 200 МБ (2 млн ключей).

Workload A, B и C:

- A (read-heavy) — 50% чтений и 50% обновлений уже существующих ключей. Обращения сконцентрированы на небольшом «горячем» подмножестве.
- B (write-heavy) — 50% последовательных вставок новых ключей и 50% чтений. В отличие от A, чтения идут не к давно популярным ключам, а преимущественно к только что вставленным в этом же потоке операций.
- C — смесь удалений, вставок, сканирования диапазонов и точечных чтений.

Сравнивались две реализации постраничного индекса:

- old — метаданные страницы в RAM
- new — описанная выше структура (B-дерево на диске, кэши в RAM)

Для новой реализации варьировался бюджет двух кэшей: кэш ключей и кэш метаданных. Во всех точках сохранялось соотношение 1:4 (ключи : метаданные), суммарная квота менялась от 0 до 2 МБ. Резидентная верхняя часть B-дерева была отключена.

В качестве основных метрик использовались:

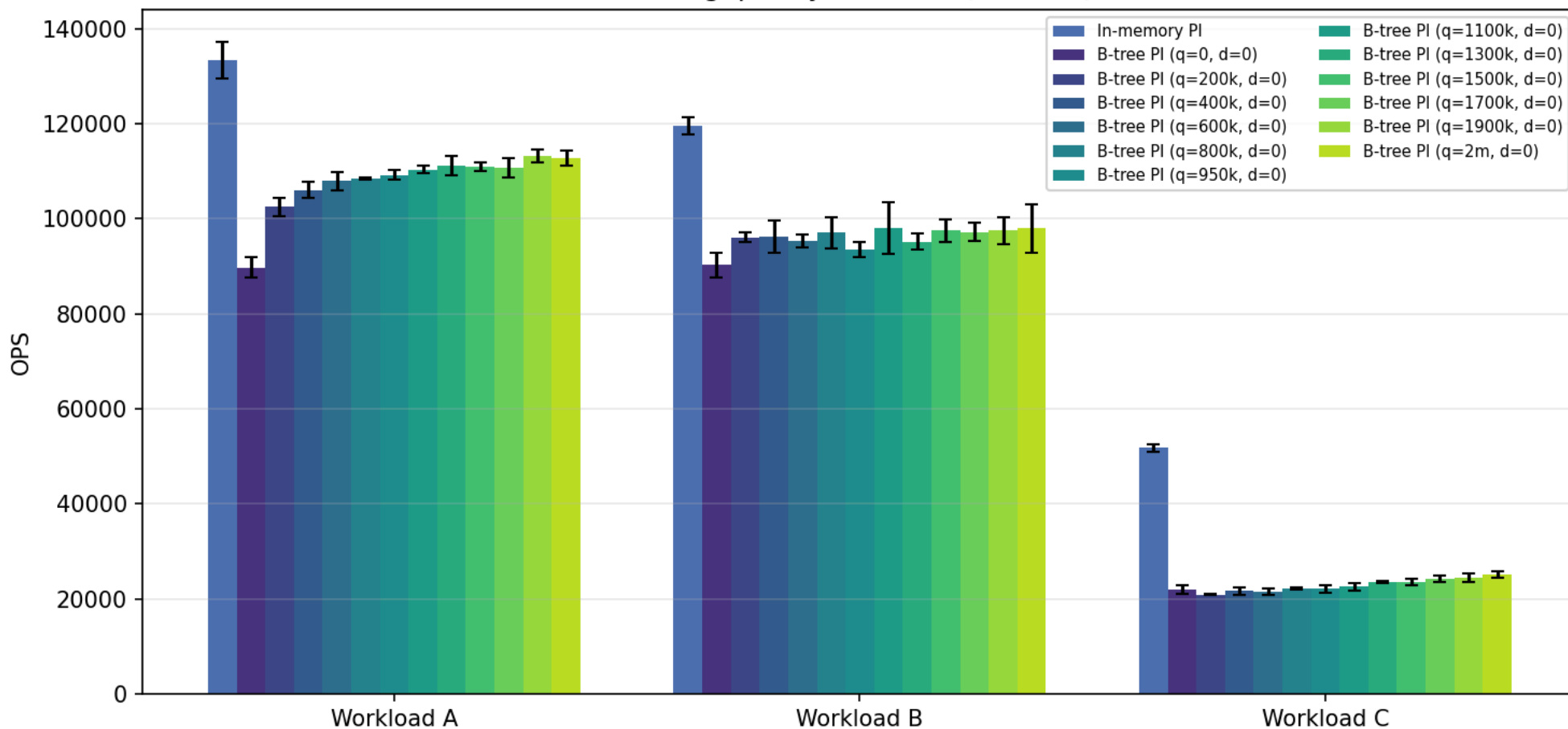
- throughput (пропускная способность) — число операций в секунду
- memory.page_index — объем памяти, занимаемой структурами постраничного индекса (фильтр Блума учитывался отдельно)
- read amplification — отношение объёма чтения с диска к объёму полезных данных, возвращённых в ответах

Дополнительно для новой реализации собиралась частота попадания в кэш (hit rate).

Пропускная способность

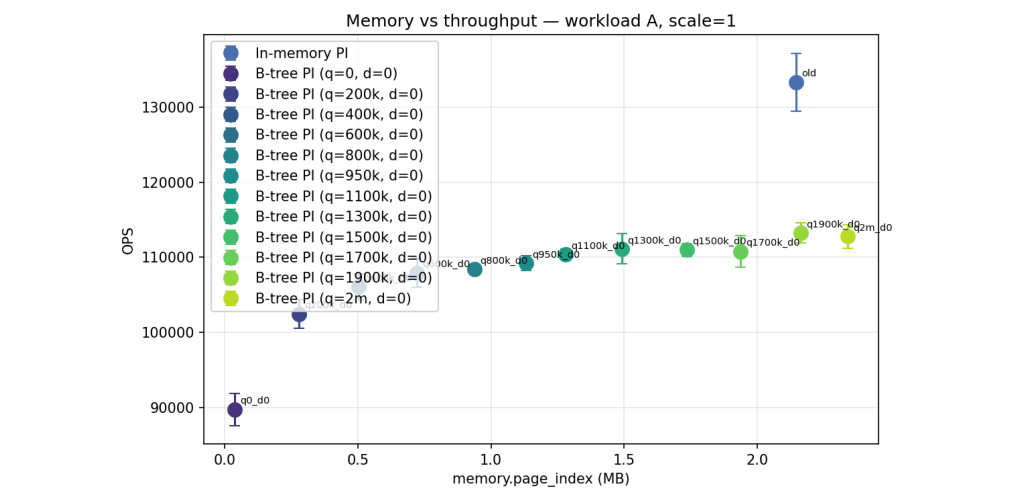


Throughput by workload (scale=1)

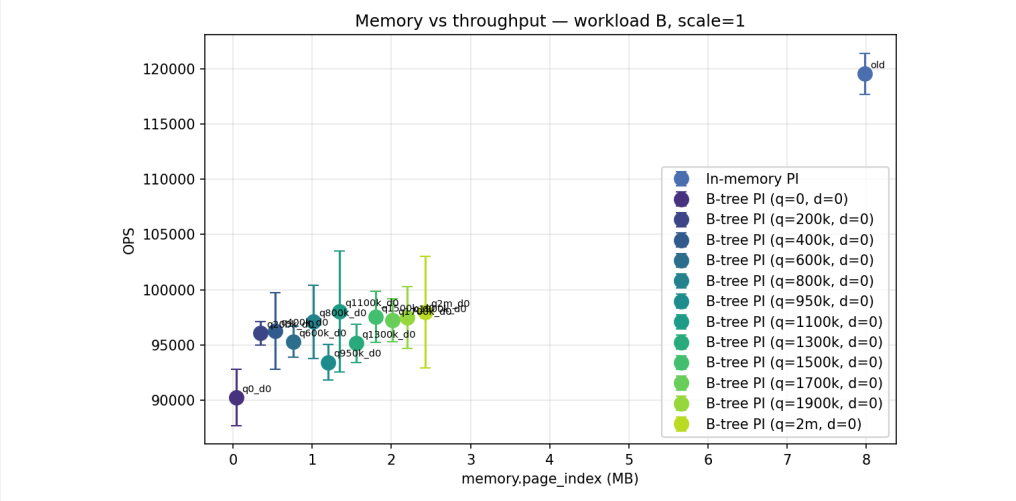


При нулевых квотах пропускная способность ниже, чем у предыдущего подхода: А – 33%, В – 25%, С – 58%. С ростом квоты новая реализация приближается к предыдущей in-мемори.

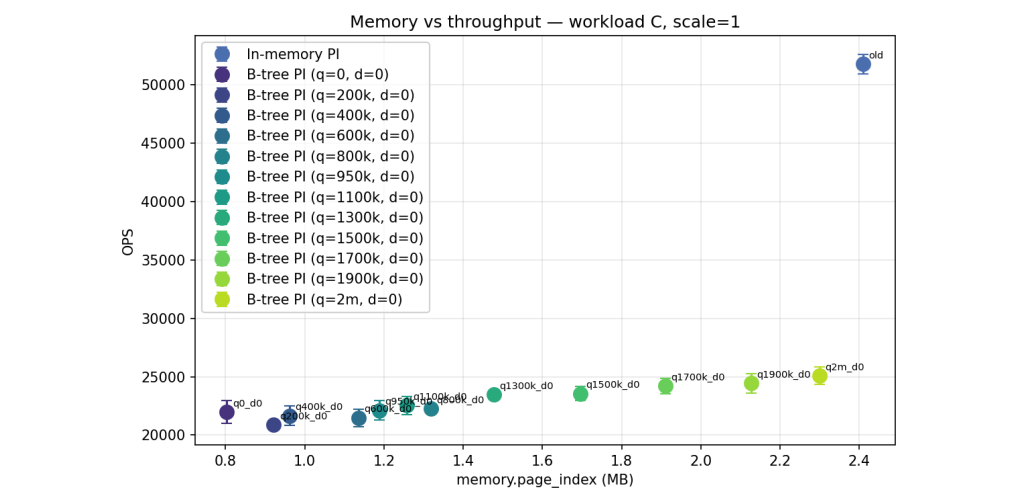
Просадка пропускной способности при снижении квоты памяти



Workload A



Workload B



Workload C

Просадка пропускной способности при снижении квоты памяти



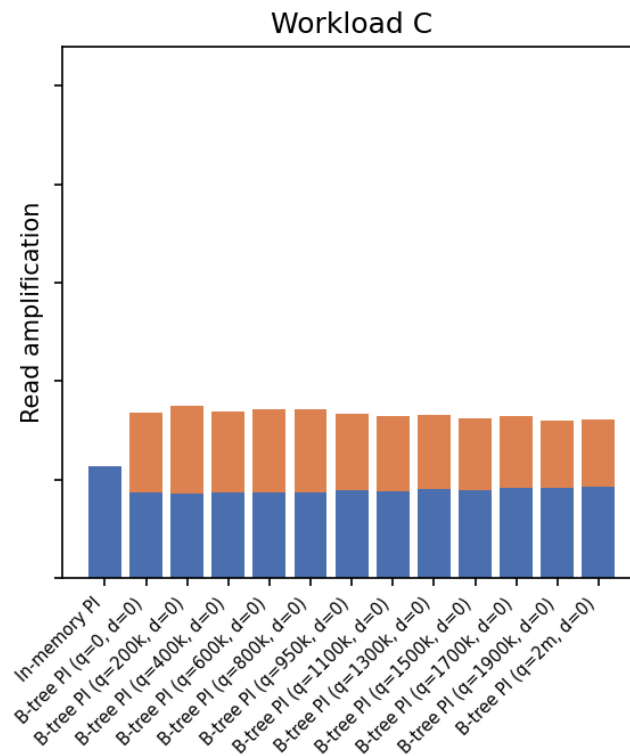
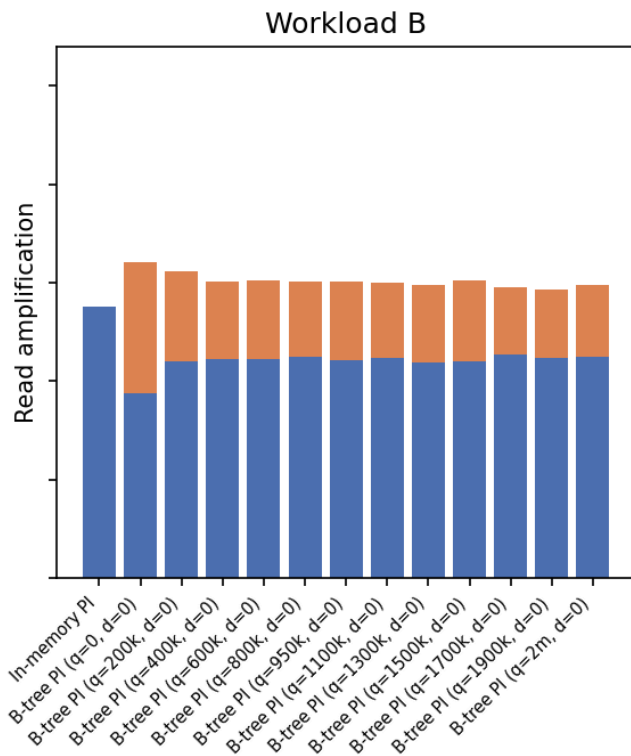
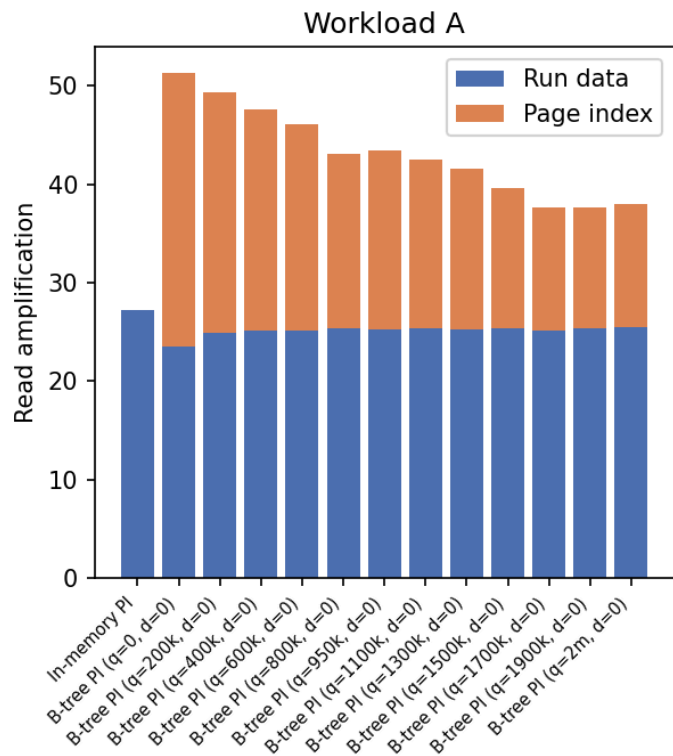
Память относительно old	Workload A	Workload B	Workload C
в 4 раза меньше	-20%	-19%	-58%
в 3 раза меньше	-19%	-18%	-58%
в 2 раза меньше	-18%	-18%	-57%

Просадка пропускной способности относительно предыдущего подхода при уменьшении памяти постраничного индекса.

Read amplification

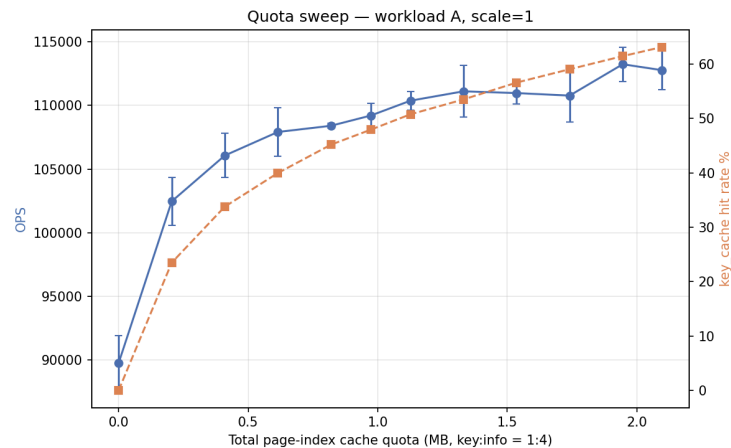


Read amplification decomposition (scale=1)



Декомпозиция: чтение данных гуп и обращения к постраничному индексу. При нулевой квоте вклад индекса максимален, с ростом квоты сжимается.

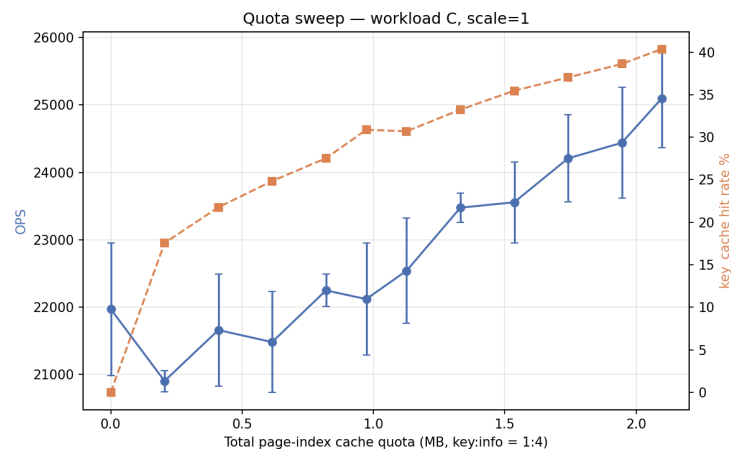
Пропускная способность, частота попадания в кэш



Workload A



Workload B



Workload C

Рост частоты попаданий в кэш согласуется с ростом пропускной способности: чем больше рабочий набор в кэшах, тем реже промахи на критическом пути.

Вывод



Новая схема постраничного индекса ограничивает обязательное потребление памяти за счёт хранения метаданных на диске и кэширования рабочего набора.

Появляется дополнительный компонент амплификация чтения (read amplification) — чтение блоков с метаданными — который заметно влияет на пропускную способность при малых квотах кэшей.